

VIBE CODING

The 6 Documents You Need Before Writing Any Code

A complete guide + ready-to-use templates for founders, indie hackers, and vibe coders.

Why these documents matter

AI coding agents are powerful — but they're not mind readers. The more context you give them upfront, the better and faster they build.

These 6 documents eliminate ambiguity. Instead of the AI guessing what stack to use, how screens connect, or where data lives — it knows. The result: fewer hallucinations, less back-and-forth, cleaner code.

Think of these as the briefing you'd give a senior developer on Day 1. The more complete the brief, the better the output.

#	Document	One-line purpose
01	PRD — Product Requirements Doc	What you're building and for whom
02	TRD — Technical Requirements Doc	Which tech, tools, and APIs you'll use
03	App Flow	Every page, every click, every navigation path
04	UI/UX Design Brief	How your app looks and feels
05	Backend Schema	Data structure, tables, auth, relationships
06	Implementation Plan	The exact step-by-step build sequence

01

PRD — Product Requirements Document

The north star. Everything else flows from this.

Why it matters

The PRD is the first document you write — before any code, design, or architecture. It defines what you're building, who it's for, and what success looks like.

Without a PRD, your AI agent fills in the blanks with assumptions. Some will be right. Most won't be. A solid PRD means fewer surprises.

What to include

- App name and tagline
- The problem you're solving and for who
- Core value proposition (what makes your app different)
- Target user persona — describe them in 2–3 sentences
- List of all features (split into Must Have / Nice to Have)
- User stories — 'As a [user], I want to [do X] so that [Y]'
- Out of scope — explicitly list what this version does NOT do
- Success metrics — how you'll know the product is working

Template

Document 01 — PRD Template	
App Name	e.g. TaskFlow
Tagline	One sentence. What does it do?
Problem	What pain does this solve? Who feels it?
Target User	Describe your primary user in 2–3 sentences
Core Features (Must Have)	List each feature on a new line
Nice to Have	Features for v2 / if time allows
Out of Scope	What this version explicitly won't do
User Stories	As a [user], I want to [action] so that [outcome]
Success Metrics	e.g. 100 signups in first week, task completion rate > 80%

02

TRD — Technical Requirements Document

The blueprint your AI agent needs to make technical decisions without guessing.

Why it matters

The TRD translates your product vision into technical decisions. It answers: what language, what framework, what database, what third-party services.

When you skip this, your AI picks a stack on its own — sometimes well, often inconsistently. The TRD locks in the choices so the agent stays coherent across every file it writes.

What to include

- Frontend framework (e.g. React, Next.js, Vue)
- Backend framework or runtime (e.g. Node/Express, Python/FastAPI)
- Database type and provider (e.g. PostgreSQL on Supabase)
- Authentication method (e.g. Supabase Auth, Clerk, NextAuth)
- Hosting and deployment (e.g. Vercel, Railway, Fly.io)
- Third-party APIs and services (e.g. Stripe, Resend, OpenAI)
- Folder structure and naming conventions
- Environment variables needed
- Any hard technical constraints or preferences

Template

Document 02 — TRD Template	
Frontend	e.g. Next.js 14 with TypeScript, Tailwind CSS
Backend	e.g. Next.js API routes / Supabase Edge Functions
Database	e.g. PostgreSQL via Supabase
Auth	e.g. Supabase Auth — email + Google OAuth
Hosting	e.g. Vercel (frontend), Supabase (backend + DB)
Third-party APIs	List each: name, purpose, free/paid tier
Key Libraries	e.g. React Query, Zod, Lucide Icons
Environment Variables	List each variable name (not value)

Constraints	e.g. Must work on mobile, must be free tier only
--------------------	--

03

App Flow — Navigation & User Journey Map

Every page, every click, every path — mapped before a single screen is built.

Why it matters

The app flow tells your AI exactly how users move through your product. What happens when they click a button. Where they go after signup. What the logged-out vs logged-in experience looks like. Without this, the AI builds pages in isolation. They work individually but don't connect. The app flow is what makes your product feel like one coherent thing.

What to include

- List of all screens / pages with short descriptions
- Navigation structure (sidebar, tabs, back buttons, etc.)
- Entry points — where does the user land first?
- Auth flow — signup → onboarding → dashboard sequence
- Key user journeys (the most important 2–3 flows)
- Edge cases — empty states, error states, loading states
- Modal / drawer / overlay interactions
- Redirect logic — where does each action take the user?

Template

Document 03 — App Flow Template	
Pages List	List every page: / (home), /login, /dashboard, /settings, etc.
Navigation Type	e.g. Top navbar, left sidebar, bottom tabs (mobile)
First Screen	What does a brand new visitor see?
Auth Flow	Signup → Email verify → Onboarding → Dashboard
Core User Journey 1	Step-by-step: User wants to [goal]. They go to... then...
Core User Journey 2	Another key flow
Empty States	What shows when there's no data yet?
Error States	e.g. Failed payment, network error — where do users go?

Redirects

e.g. After login → /dashboard, after logout → /

04

UI/UX Design Brief — Visual & Interaction Design Guide

So the AI doesn't just build something functional — it builds something you'd actually use.

Why it matters

AI agents default to plain, unstyled, or inconsistent UIs when left to their own judgment. The design brief locks in your visual language: colors, fonts, component style, spacing, and overall vibe. You don't need to be a designer. Even a rough direction ('dark mode, minimal, like Linear') gives the AI enough to make coherent decisions across every screen.

What to include

- Overall aesthetic direction (e.g. minimal, bold, playful, corporate)
- Color palette — primary, secondary, background, text, accent
- Typography — font choices, heading sizes, body size
- Component style — rounded vs sharp corners, flat vs shadowed
- Dark mode / light mode preference
- Inspiration references (apps you like the look of)
- Key UI patterns (cards, tables, modals, sidebars)
- Mobile responsiveness requirements
- Accessibility considerations (contrast, font size, etc.)

Template

Document 04 — UI/UX Design Brief Template	
Aesthetic	e.g. Minimal, clean, dark-mode first. Like Linear or Vercel.
Primary Color	e.g. #6C47FF (indigo)
Background Color	e.g. #0D0D0D (dark) or #FFFFFF (light)
Text Color	e.g. #F5F5F7
Accent / CTA Color	e.g. #6C47FF
Font	e.g. Inter for UI, Geist Mono for code
Border Radius	e.g. 8px rounded corners throughout

Shadows	e.g. Subtle card shadows, no heavy drop shadows
Dark/Light Mode	e.g. Dark mode primary, light mode optional
Reference Apps	e.g. Linear, Vercel, Notion, Raycast
Mobile	e.g. Fully responsive, bottom tab nav on mobile

05

Backend Schema — Data Model & Auth Architecture

How your data is stored, structured, and secured — defined before the AI writes a single migration.

Why it matters

The backend schema is the hardest document to change after the fact. Once your database has data in it, restructuring tables is painful. Getting this right upfront saves hours of debugging and migration hell.

Your AI agent uses this to write database migrations, API routes, and auth logic. The more precise you are, the less it has to guess — and the fewer security holes end up in your codebase.

What to include

- List of all database tables with columns and data types
- Primary keys and foreign key relationships
- Indexes (which columns need fast lookup?)
- Auth model — who can access what? (Row Level Security rules)
- User roles and permissions (admin, user, guest, etc.)
- Any sensitive fields that need encryption
- File/media storage structure (if applicable)
- Webhooks or event triggers
- API endpoint list (optional but helpful)

Template

Document 05 — Backend Schema Template	
Table: users	id (uuid), email (text), name (text), created_at (timestamp), role (text)
Table: [your table]	id, [columns], user_id (FK → users.id), created_at
Table: [your table]	Add more tables as needed
Relationships	e.g. projects.user_id → users.id (many-to-one)
Auth Provider	e.g. Supabase Auth — JWT tokens, email/OAuth
Row Level Security	e.g. Users can only read/write their own rows

User Roles	e.g. admin: full access, user: own data only
File Storage	e.g. Supabase Storage — /avatars/{user_id}, /uploads/{user_id}
Sensitive Fields	e.g. payment_method — stored via Stripe, not in DB

06

Implementation Plan — Step-by-Step Build Sequence

The exact order to build so the AI never skips a foundation layer.

Why it matters

The implementation plan is your build roadmap. It tells the AI agent what to build first, second, and third — and why that order matters. Skipping this leads to the AI building the frontend before the backend exists, or adding auth after the DB schema is half-baked.

Think of it as a checklist the AI ticks off. Clear phases, clear milestones, clear completion criteria. The more specific you are, the less the agent freelances.

What to include

- Numbered phases (not just tasks — phases with goals)
- Phase 1 always: Project setup, repo, folder structure, env vars
- Phase 2: Database schema and migrations
- Phase 3: Authentication flow
- Phase 4: Core features (in order of dependency)
- Phase 5: UI polish and responsive design
- Phase 6: Testing, error handling, edge cases
- Phase 7: Deployment and environment configuration
- Explicit 'done' criteria for each phase

Template

Document 06 — Implementation Plan Template	
Phase 1: Setup	Init project, install deps, configure env vars, set up repo structure
Phase 2: Database	Write all migrations, set up RLS policies, seed test data
Phase 3: Auth	Implement signup/login/logout, protected routes, session handling
Phase 4: Core Feature 1	Describe the feature and its sub-tasks in order
Phase 5: Core Feature 2	Next most important feature

Phase 6: Core Feature 3	Continue for all core features
Phase 7: UI Polish	Responsive design, loading states, empty states, error states
Phase 8: Testing	Manual test all flows, fix edge cases, handle errors gracefully
Phase 9: Deploy	Set up prod environment, configure domains, final env vars
Done Criteria	What does 'finished' look like? e.g. All core flows work end-to-end

How to use these documents

You don't have to write all 6 from scratch manually. Use Claude or ChatGPT to help you fill them out — just paste the template, answer the prompts conversationally, and let the AI clean it up into a proper document.

Suggested workflow

1. Start with the PRD — get your product idea out of your head and into writing
2. Fill in the TRD — decide your stack before anything else
3. Map your App Flow — draw it out, then describe it in text
4. Write the UI/UX brief — even just 'dark mode, minimal, like Notion' helps
5. Design your Backend Schema — tables first, relationships second
6. Write your Implementation Plan — phases, not just a flat task list

Once all 6 are done, paste them at the start of your AI agent conversation. Tell it: 'Here are my project documents. Use these as the source of truth for everything you build.'

The 10 minutes you spend on these documents saves hours of fixing broken code, re-explaining context, and undoing bad architectural decisions.